

Why We Need a Library/Framework

Angular vs React vs Vue — quick reference

WHY NOT JUST PLAIN JAVASCRIPT?

Problem with plain JS	What a framework gives you
Manually updating the DOM on every data change	Reactive rendering — UI updates automatically
No standard way to structure growing code	Component-based architecture
Re-writing routing, forms, state every project	Built-in tooling for routing, state, HTTP, forms
Hard to keep large teams consistent	Shared conventions & ecosystem
Re-rendering whole pages hurts performance	Virtual DOM / fine-grained reactivity
Messy testing without structure	Components are easy to unit test in isolation

Bottom line: frameworks solve the scaling problem. Plain JS is fine for a simple page; once you have dynamic data, many UI states, and a growing team, you need structure.

ANGULAR vs REACT vs VUE

	Angular	React	Vue
Type	Full framework	Library (UI only)	Framework (progressive)
Language	TypeScript-first	JS/TS, JSX	JS/TS, templates
Learning curve	Steepest	Moderate	Easiest
Structure	Opinionated — DI, routing, forms, HTTP all built-in	Unopinionated — you pick router, state lib, etc.	Middle ground — official router/state libs, flexible
State mgmt	RxJS / Services built-in	External lib (Redux, Zustand) or built-in state	Built-in reactivity + Pinia/Vuex
Best for	Large enterprise apps, team consistency	Flexibility, huge ecosystem, most jobs	Small-mid apps, fast to learn, great DX
Backed by	Google	Meta	Community (Evan You)

ONE-LINE TAKEAWAYS

- **Angular** = “batteries included,” opinionated — good for big enterprise teams
- **React** = flexible library + huge ecosystem — you assemble your own stack
- **Vue** = gentle learning curve, great developer experience, progressive adoption